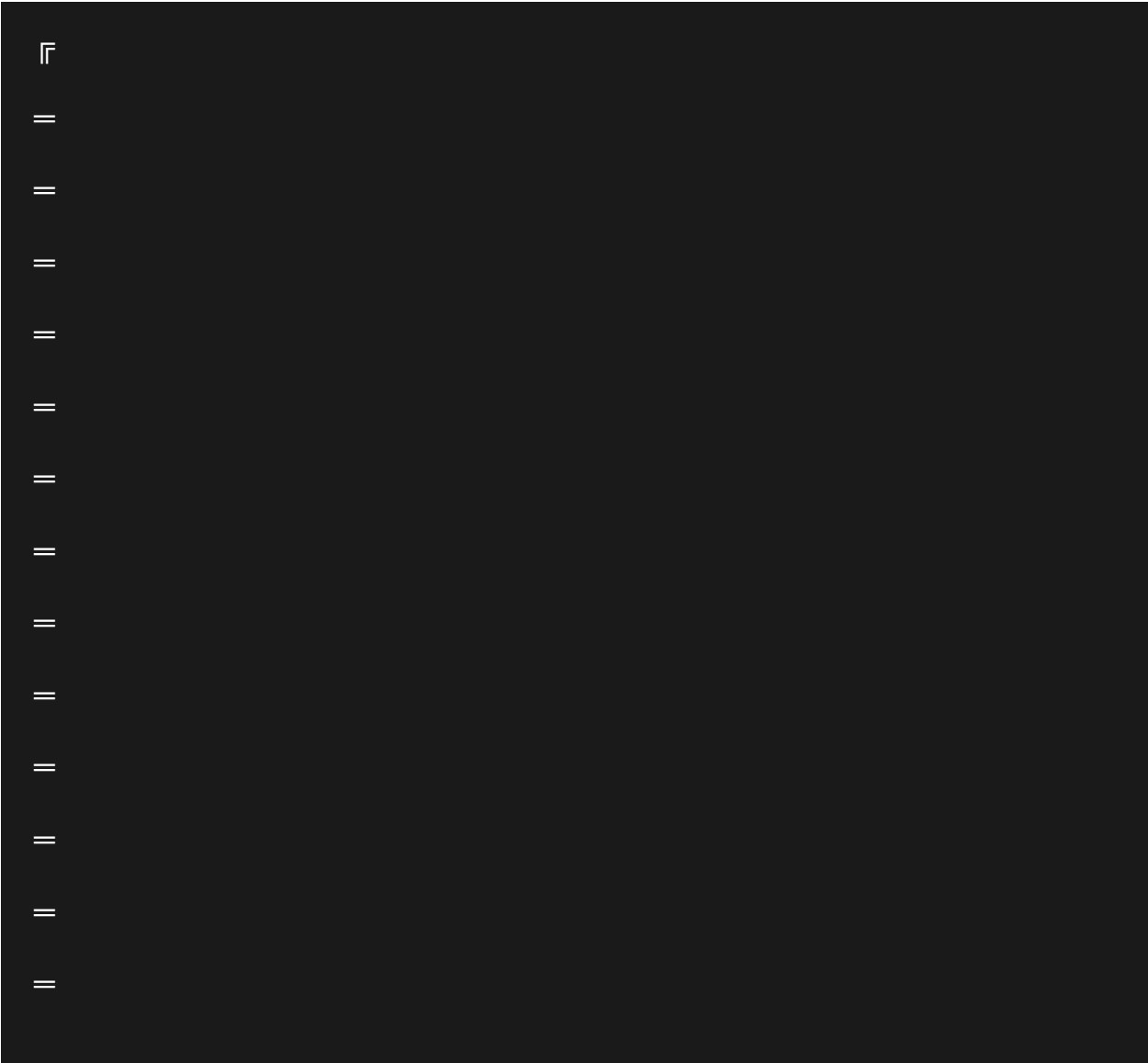

Multi-Tag Agent System

Developer README v1.0 — Everything you need to clone, configure, run, and extend the system.

Version: 1.0 | Date: June 17, 2026 | Repo: github.com/auron/multi-tag-agent (internal) | Status: Production-Ready



$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$

$$=$$

$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$

$$=$$

=

=

=

=

=

=

7

||

MULTI-TAG AGENT SYSTEM

.

Developer README

||

||

One system. Multiple minds. Zero context loss.

||

ℒ

=

=

=

=

=

=

$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$

$$=$$

$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$
$$=$$

$$=$$

=

=

=

=

=

=

=

=

=

=

=

=

=

』

[Python 3.11+]

[License: MIT]

[API: REST/JSON]

[LLM: Provider-Agnostic]

[Tests: 100+ Cases]

[Status: Production-Ready]

The **Multi-Tag Agent System** is a modular, context-driven AI agent architecture that organizes LLM interactions into isolated prompt containers called **Tags**. Each Tag maintains its own persona, behavioral rules, and task focus. A natural-language **Context Router** dispatches user input to the correct Tag automatically — no UI switches, no slash commands. A persistent **Identity Alias Layer (IAL)** ensures user context travels seamlessly across all Tag switches, so nothing is ever lost between modes.

Table of Contents

- 1. Prerequisites
 - 2. Quick Start (5 Minutes)
 - 3. Project Structure
 - 4. Configuration Reference
 - 5. Tag System — Using & Authoring Tags
 - 6. Amplifier System
 - 7. API Reference — Key Endpoints with cURL Examples
 - 8. Authentication
 - 9. Running the Test Suite
 - 10. Deployment
 - 11. Architecture Overview
 - 12. Extending the System
 - 13. Troubleshooting
 - 14. Contributing
 - 15. Document Suite
 - 16. License
-

1. Prerequisites

Required

Dependency	Minimum Version	Purpose
Python	3.11+	Backend runtime
Redis	7.0+	Session state & short-term persistence
PostgreSQL	15+	IAL profiles & long-term persistence
Docker & Docker Compose	24+	Local development environment
LLM API Key	—	OpenAI / Azure OpenAI / Anthropic Claude

Optional (but Recommended)

Tool	Purpose
<code>httpx</code>	Python HTTP client for SDK usage
<code>Locust</code>	Load testing (TC-PERF suite)
<code>pytest</code>	Test suite execution
<code>pre-commit</code>	Code quality hooks

2. Quick Start (5 Minutes)

Step 1 — Clone the Repository

```
git clone https://github.com/auron/multi-tag-agent.git cd multi-tag-agent
```

Step 2 — Copy and Configure Environment

```
cp .env.example .env
```

Edit `.env` with your values:

```
# .env LLM_BACKEND=openai # openai | azure |  
anthropic | mock OPENAI_API_KEY=sk-... # Your LLM API  
key DATABASE_URL=postgresql://user:pass@localhost:5432/multitag  
REDIS_URL=redis://localhost:6379/0 JWT_SECRET=your-secure-secret-here  
# Min 32 chars LOG_LEVEL=INFO
```

Step 3 — Start with Docker Compose

```
docker-compose up -d
```

This starts: **API server** (port 8000), **PostgreSQL** (port 5432), **Redis** (port 6379).

Step 4 — Verify Health

```
curl http://localhost:8000/v1/health
```

Expected response:

```
{  "status": "healthy",  "components": {    "api": "healthy",  
  "context_router": "healthy",    "llm_backend": "healthy",    "redis":  
  "healthy",    "database": "healthy"  },  "version": "1.0.0" }
```

Step 5 — Create Your First Session and Send a Message

```
# 1. Create an IAL profile curl -X POST http://localhost:8000/v1/ial/profiles  
\ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json"  
\ -d '{  "alias": "Auron",    "communication_style": "direct",  
  "preferred_output_format": "structured",    "language": "en"  }' # 2.  
Create a session curl -X POST http://localhost:8000/v1/sessions \ -H  
"Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d  
'{  "alias": "Auron",    "default_tag": "auron"  }' # → save the  
returned session_id # 3. Send your first message curl -X POST  
http://localhost:8000/v1/sessions/$SESSION_ID/messages \ -H "Authorization:  
Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d  
'{  "user_input": "Auron, I need a plan for the project week."  }'
```

✓ TIP:

You are up and running. Total time: approximately 5 minutes. Use

```
docker-compose logs -f api
```

to watch live log output during your first session.

3. Project Structure

```

multi-tag-agent/ | | config/ # YAML configuration files
| | tags.yaml # Tag definitions (system + custom) | |
amplifiers.yaml # Amplifier definitions | | ial.yaml
# IAL global defaults | | src/ # Application source
| | api/ # FastAPI route handlers | | |
sessions.py # POST/GET/DELETE /sessions | | | tags.py
# CRUD /tags | | | messages.py # POST /messages (core router) |
| | | ial.py # CRUD /ial/profiles | | | amplifiers.py
# /amplifiers | | | snapshots.py # /snapshots | | |
health.py # GET /health, /diagnostics | | | core/
# Core system components | | | router.py # Context Router &
Tag Dispatcher | | | tag_manager.py # Tag registry & lifecycle |
| | | ial_manager.py # IAL profile management | | |
amplifier_engine.py # Amplifier overlay logic | | | session_manager.py
# Session lifecycle & state | | | persistence.py # Redis +
PostgreSQL interface | | | llm/ # LLM backend
abstraction | | | base.py # Abstract LLM client | | |
openai_client.py # OpenAI / Azure OpenAI | | | anthropic_client.py #
Anthropic Claude | | | mock_client.py # Deterministic mock
(testing) | | | middleware/ # FastAPI middleware | | |
auth.py # JWT & API key validation | | | injection_guard.py
# Prompt injection detection | | | rate_limiter.py # Rate limiting
| | | models/ # Pydantic data models | | | session.py
| | | tag.py | | | message.py | | | ial.py | | |
amplifier.py | | | snapshot.py | | | main.py #
FastAPI app entry point | | tests/ # Full test suite
(100+ cases) | | | unit/ | | | integration/ | | | e2e/ | | |
security/ | | | performance/ | | | contract/ | | | fixtures/ | | |
conftest.py | | | scripts/ # Utility scripts | | |
seed_tags.py # Load tags.yaml into DB | | | generate_token.py
# Generate test JWT | | | migrate.py # Database migrations |
| | | docs/ # Companion document suite | | |
architecture-spec-v1.0.docx | | | api-blueprint-v1.0.docx | | | test-
suite-spec-v1.0.docx | | | whitepaper-v1.0.docx | | | .env.example
# Environment template | | | docker-compose.yml # Local dev stack |
Dockerfile # API container | | | pyproject.toml #
Python project config | | requirements.txt # Production
dependencies | | requirements-dev.txt # Dev + test dependencies |
README.md # This file

```

4. Configuration Reference

4.1 Environment Variables

Variable	Required	Default	Description
LLM_BACKEND	Yes	—	LLM provider: openai, azure, anthropic, mock
OPENAI_API_KEY	If openai	—	OpenAI API key

Variable	Required	Default	Description
AZURE_OPENAI_KEY	If azure	—	Azure OpenAI API key
AZURE_OPENAI_ENDPOINT	If azure	—	Azure endpoint URL
ANTHROPIC_API_KEY	If anthropic	—	Anthropic Claude API key
DATABASE_URL	Yes	—	PostgreSQL connection string
REDIS_URL	Yes	—	Redis connection string
JWT_SECRET	Yes	—	HS256 signing secret, min 32 chars
JWT_EXPIRY_SECONDS	No	3600	JWT token lifetime
DEFAULT_SESSION_TTL	No	86400	Default session lifetime in seconds
MAX_INPUT_LENGTH	No	32768	Max user_input character length
MAX_SYSTEM_PROMPT_LENGTH	No	8192	Max Tag system prompt length
RATE_LIMIT_RPM	No	60	Requests per minute per API key
LOG_LEVEL	No	INFO	DEBUG, INFO, WARNING, ERROR
ENABLE_AUDIT_LOG	No	true	Write Tag-switch events to audit log
INJECTION_GUARD_ENABLED	No	true	Enable prompt injection detection

4.2 Tag Configuration (config/tags.yaml)

```
# config/tags.yaml #
# System Tags
are pre-loaded and immutable. # Custom Tags can be added below or via API. #
tags: # —
SYSTEM TAG: AURON WORKFLOW _____ auron: name:
"Auron Workflow" type: system is_default: true #
Fallback tag when no trigger detected isolation_level: high
# high | full amplifier_compatible: true # Allow Dickerchen
overlay output_format: structured # structured | freeform |
technical verbosity: medium # low | medium | high
task_tracking: true # Append [STATUS:... ] to responses
trigger_keywords: - plan - task - schedule - workflow
- organize system_prompt: | You are a highly specialized workflow
assistant for Auron. ALIAS: Auron (immutable, used throughout the
dialogue) FOCUS: Task structuring | Workflow optimization | Time
savings COMMUNICATION: Direct, precise, structured. No pleasantries.
```

```

OUTPUT FORMAT: Numbered steps, time estimates, clear prioritization.
CONTEXT ISOLATION: Ignore content from other Tags, except IAL data.
NEGATIVE CONSTRAINTS: - No roleplay - Do not simulate
emotional bonds - Do not violate security policies PERSISTENCE:
Append [STATUS: ...] snapshot to every response. # — SYSTEM TAG: DEEP-
DIVE TECHNICAL _____ deep_dive: name: "Deep-Dive
Technical" type: system is_default: false isolation_level: full
# Full isolation - no cross-tag bleeding amplifier_compatible: true
output_format: technical verbosity: high task_tracking: false
trigger_keywords: - code - architecture - debug -
deep-dive - review - analyze system_prompt: | You are a
technical depth analyst. No small talk. FOCUS: Code | Architecture |
Systems | Debugging | Concepts COMMUNICATION: Technically precise. Cite
sources. Use code blocks. REFERENCE LEVEL: Senior Engineer / System
Architect NEGATIVE CONSTRAINTS: - No simplifying analogies when
user is technical - No speculation without [SPEC] tag # — CUSTOM
TAG EXAMPLE _____ # creative: # name:
"Creative Mode" # type: custom # is_default: false #
isolation_level: high # amplifier_compatible: true # output_format:
freeform # verbosity: high # trigger_keywords: # - brainstorm
# - ideas # - creative # system_prompt: | # You are a
creative ideation assistant for Auron...

```

4.3 Amplifier Configuration (config/amplifiers.yaml)

```

# config/amplifiers.yaml amplifiers: # — BUILT-IN: DICKERCHEN
_____ dickerchen: keyword: "dickerchen"
# Case-insensitive trigger type: system persistence: single-shot
# single-shot | session depth_multiplier: 2.0 # Output depth
factor add_alternatives: true # Always include alternative
approaches add_risks: true # Always include risks and
edge cases output_suffix: "[AMPLIFIER ADDITION: {content}]"
overlay_prompt: | AMPLIFIED MODE ACTIVE. Increase output depth by
2x. Add: alternative approaches, risks, edge cases. Confirm
activation with: "// Amplifier active" End response with: [AMPLIFIER
ADDITION: <summary of additions>] # — CUSTOM AMPLIFIER EXAMPLE
_____ # turbo: # keyword: "turbo" # type:
custom # persistence: single-shot # depth_multiplier: 0.5 #
Halve depth for max brevity # add_alternatives: false # add_risks:
false # overlay_prompt: | # TURBO MODE: Maximum brevity. Bullet
points only. No elaboration.

```

5. Tag System — Using & Authoring Tags

5.1 Using Tags via Natural Language

Tags respond to natural-language triggers. No special syntax required:

What You Type	What Happens
"Auron, I need a release plan"	Auron Workflow Tag activates (explicit

What You Type	What Happens
	trigger)
"I need to organize this week"	Auron activates via keyword "organize" (implicit)
"Deep-Dive: review my schema"	Deep-Dive Tag activates (explicit trigger)
"debug this function"	Deep-Dive activates via keyword "debug" (implicit)
"hello"	Default Tag (Auron) responds — no trigger needed
"Dickerchen, analyze risks"	Current Tag stays active; Amplifier overlays it
"Dickerchen, Deep-Dive: analyze"	Deep-Dive activates AND Amplifier overlays simultaneously

5.2 Tag Switching Rules

Critical behavior every developer must understand before integrating:

RULE 1: Explicit trigger > Implicit trigger > Default
 RULE 2: Amplifier (Dickerchen) is NOT a tag switch – it is an overlay
 RULE 3: Every switch saves current tag state (context snapshot)
 RULE 4: Switching back restores the snapshot – zero context loss
 RULE 5: Conflict (two tags triggered simultaneously) → user is asked to clarify

5.3 Authoring a Custom Tag

Method A — YAML (no code required)

1. Add your Tag block to config/tags.yaml (see Section 4.2 template)
2. Run the seed script: `python scripts/seed_tags.py`
3. Restart the API: `docker-compose restart api`

Method B — via API

```
curl -X POST http://localhost:8000/v1/tags \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "tag_id": "creative",
    "name": "Creative Mode",
    "system_prompt": "You are a creative ideation assistant for Auron. Focus on brainstorming and lateral thinking...",
    "trigger_keywords": ["brainstorm", "ideas", "creative", "imagine"],
    "isolation_level": "high",
    "amplifier_compatible": true,
    "output_format": "freeform",
    "verbosity": "high"
  }'
```

Tag Authoring Checklist

4. Define a unique `tag_id` (lowercase, underscores only, max 32 chars)

5. Write a focused `system_prompt` — one job per Tag, no overlapping responsibilities
6. Choose `isolation_level: full` for Tags that must never bleed into others (technical, sensitive)
7. Select at least 3 `trigger_keywords` that are semantically distinct from existing Tags
8. Run conflict check: `POST /diagnostics/route` with each keyword to verify no conflicts
9. Test with fixture-based dialogues before activating in production
10. Never put security-bypassing instructions in a custom Tag prompt — these are rejected server-side

 WARNING — Tag Prompt Security:

Custom Tag prompts are validated server-side on creation. Prompts containing injection patterns, self-referential loops, or instructions to bypass security policies are rejected with

PROMPT_INJECTION_DETECTED (400)

. The

`system_prompt`

field is not a general-purpose prompt injection vector.

6. Amplifier System

6.1 How Amplifiers Work

The Amplifier is a **single-shot overlay** — it deepens the current Tag's output for exactly one response, then auto-deactivates:

```
Normal mode:      User:  "Auron, give me the plan."      Agent:
[Concise structured plan] Amplified mode: User:  "Dickerchen, give me the
plan."            Agent: // Amplifier active | Base: Auron
[Structured plan]                    [AMPLIFIER ADDITION:
Risks: ...                        Alternatives: ...
Edge Cases: ...] Next message:    Amplifier is OFF. Back to normal mode.
```

6.2 Combining Amplifier + Tag Switch

```
# Trigger Deep-Dive AND amplify in one message: "Dickerchen, Deep-Dive:
analyze my database schema." # Result: # → Tag switched to:    deep_dive #
→ Amplifier overlay:    active # → Output:                    Technical schema
analysis at 2x depth #                                     + benchmarks, alternative
schemas, literature [AMS]
```

6.3 Manual Amplifier Trigger via API

Pre-queue an amplifier for the next message programmatically:

```
curl -X POST
http://localhost:8000/v1/sessions/$SESSION_ID/amplifiers/dickerchen/trigger \
-H "Authorization: Bearer $TOKEN"
```

Response:

```
{  "amplifier_id": "dickerchen",    "status": "queued",
  "applies_to_next_message": true }
```

INFO:

Only one Amplifier may be active at a time (stack depth: 1). Attempting to trigger a second Amplifier before the first fires will return

AMPLIFIER_STACK_OVERFLOW (409)

. Deactivate the queued Amplifier before queuing another.

7. API Reference — Key Endpoints with cURL Examples

All examples below assume environment variables are set:

`BASE=http://localhost:8000/v1`, `TOKEN`, and `SESSION_ID`.

Core Flow — The 5 Most Important Calls

1. Create IAL Profile

```
curl -X POST $BASE/ial/profiles \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{      "alias": "Auron",      "communication_style": "direct",      "preferred_output_format": "structured",      "language": "en",      "timezone": "Europe/Berlin"    }'
```

2. Create Session

```
curl -X POST $BASE/sessions \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{      "alias": "Auron",      "default_tag": "auron",      "ial_profile_id": "profile_01HXYZ",      "ttl_seconds": 86400    }'
```

3. Send Message (Auto-Routing)

```
curl -X POST $BASE/sessions/$SESSION_ID/messages \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{      "user_input": "Deep-Dive: What are the trade-offs of event sourcing vs CRUD?"    }'
```

4. Send Message with Streaming

```
curl -X POST $BASE/sessions/$SESSION_ID/messages \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -H "Accept: text/event-stream" \ -d '{      "user_input": "Auron, plan my sprint for this week.",      "stream": true    }'
```

5. Test Router Without LLM Call (Dry Run)

```
curl -X POST $BASE/diagnostics/route \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{      "user_input": "Dickerchen, Deep-Dive: analyze my API schema.",      "current_tag": "auron"    }' # Expected: detected_tag=deep_dive, amplifier_detected=true, routing_confidence >= 0.95
```

Additional Useful Calls

Activate Tag Explicitly

```
curl -X POST $BASE/sessions/$SESSION_ID/tags/deep_dive/activate \ -H "Authorization: Bearer $TOKEN"
```

Create Manual Snapshot

```
curl -X POST $BASE/sessions/$SESSION_ID/snapshots \ -H "Authorization: Bearer $TOKEN" \ -H "Content-Type: application/json" \ -d '{"tag_id": "auron", "label": "pre-release checkpoint"}'
```

Restore a Snapshot

```
curl -X POST $BASE/sessions/$SESSION_ID/snapshots/$SNAPSHOT_ID/restore \ -H "Authorization: Bearer $TOKEN"
```

Close Session

```
curl -X DELETE $BASE/sessions/$SESSION_ID \ -H "Authorization: Bearer $TOKEN" # → 204 No Content; final snapshot auto-created
```

8. Authentication

8.1 Generate a Development Token

```
python scripts/generate_token.py \ --subject "dev-user-001" \ --scopes "agent:read agent:write amplifier:use" \ --expiry 3600
```

8.2 Token Scopes

Scope	Grants Access To
agent:read	GET on sessions, tags, messages, snapshots
agent:write	POST/PUT/PATCH/DELETE on sessions, tags, messages
ial:read	GET on IAL profiles
ial:write	POST/PATCH/DELETE on IAL profiles
amplifier:use	Trigger amplifiers; POST on amplifiers
admin:all	Audit log; Tag system prompt inspection; all routes

8.3 Service-to-Service (API Key)

```
curl -X GET $BASE/sessions \ -H "X-API-Key: your_service_api_key"
```

8.4 Common Auth Errors

Error	HTTP Status	Cause
INVALID_TOKEN	401	Missing, malformed, or expired JWT
IAL_WRITE_FORBIDDEN	403	Caller lacks ial:write scope
TAG_ISOLATION_BREACH	403	Attempted cross-Tag access
RATE_LIMIT_EXCEEDED	429	> 60 requests/minute per key

9. Running the Test Suite

9.1 Install Test Dependencies

```
pip install -r requirements-dev.txt
```

9.2 Run All Tests (Mock LLM, Local)

```
# Start test dependencies docker-compose -f docker-compose.test.yml up -d #
Run full suite pytest tests/ -v # Run with coverage report pytest tests/ --
cov=src --cov-report=html open htmlcov/index.html
```

9.3 Run Tests by Layer

```
# Unit tests only (fastest – no network) pytest tests/unit/ -v # Integration
tests (requires running API + Redis + DB) pytest tests/integration/ -v #
End-to-end workflow tests pytest tests/e2e/ -v # Security tests pytest
tests/security/ -v # Performance tests (requires Locust) locust -f
tests/performance/locustfile.py --headless \ -u 50 -r 10 --run-time 60s \
--host http://localhost:8000
```

9.4 Run a Single Test Case by ID

```
# Run TC-UNIT-012 (Prompt injection detection) pytest
tests/unit/test_context_router.py::test_prompt_injection_detection -v # Run
TC-INT-026 (Amplifier triggered in message) pytest
tests/integration/test_messages_api.py::test_amplifier_triggered -v # Run
TC-E2E-001 (Full project planning workflow) pytest
tests/e2e/test_workflow_project_planning.py::test_full_planning_workflow -v
```

9.5 Coverage Targets (CI Enforcement)

Priority	Test Layer	Required Pass Rate
P0 - Blocker	Unit + Integration + E2E + Security	100%

Priority	Test Layer	Required Pass Rate
P1 - Critical	All layers	95%
P2 - High	All layers	80%
Overall branch coverage	src/	≥ 85%

⚠ WARNING — CI Gate:

The CI pipeline blocks merge if any P0 test fails or if overall branch coverage in src/ drops below 85%. There are no exceptions — fix tests before raising a PR.

10. Deployment

10.1 Local Development

```
docker-compose up -d # API available at: http://localhost:8000 # Docs
(OpenAPI): http://localhost:8000/docs
```

10.2 Production — Docker

```
# Build production image docker build -t multi-tag-agent:1.0.0 . # Run with
production env docker run -d \ --name multi-tag-agent \ -p 8000:8000 \
--env-file .env.production \ multi-tag-agent:1.0.0
```

10.3 Cloud Deployment Options

Platform	Method	Notes
Azure Container Apps	Push image → ACA	Managed scaling; pair with Azure Redis Cache + Azure PostgreSQL
AWS Lambda	Mangum ASGI adapter	Cold-start latency: ~300ms; use provisioned concurrency for p95 targets

Platform	Method	Notes
Kubernetes	Helm chart (see /deploy/helm/)	Horizontal pod autoscaler on CPU; Redis Cluster for HA
Railway / Render	One-click Docker deploy	Good for staging; not recommended for enterprise production

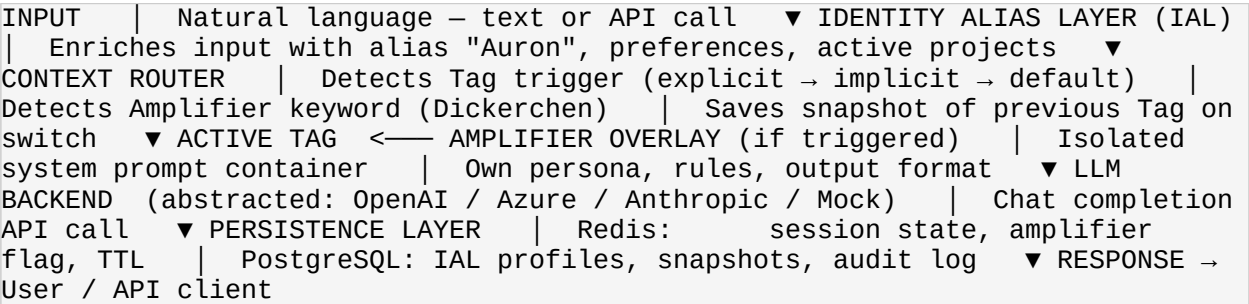
10.4 Database Migrations

```
# Run migrations (Alembic) python scripts/migrate.py upgrade head # Generate new migration after model changes alembic revision --autogenerate -m "describe change"
```

10.5 Seed System Tags

```
# Load tags.yaml and amplifiers.yaml into database python scripts/seed_tags.py --env production
```

11. Architecture Overview



See Also:

For the complete architecture deep-dive — prompt stacks, isolation model, CAG (Context Aggregation Graph), and AMS (Artifact Management System) — refer to docs/architecture-spec-v1.0.docx

.

12. Extending the System

12.1 Adding a New LLM Backend

11. Create `src/llm/your_provider_client.py` extending `BaseLLMClient`
12. Implement `chat(system, messages, user) → str` and `stream(...)` methods
13. Register in `src/llm/__init__.py` factory
14. Set `LLM_BACKEND=your_provider` in `.env`

```
# src/llm/base.py class BaseLLMClient(ABC):      @abstractmethod      async def
chat(self, system: str, messages: list, user: str) -> str: ...
@abstractmethod      async def stream(self, system: str, messages: list, user:
str): ...
```

12.2 Adding a New Router Detection Strategy

The Context Router supports pluggable detection strategies:

```
# src/core/router.py class ContextRouter:      def detect_tag(self, input:
str, current_tag: str) -> RoutingDecision:      # Step 1: Explicit trigger
detection (keyword match)      # Step 2: Implicit trigger (embedding
similarity – plug in here)      # Step 3: Default fallback      ...
```

To swap in semantic embedding-based matching (planned for v1.1), replace Step 2 with your embedding strategy. The `RoutingDecision` return type is stable across implementations.

12.3 Custom Amplifiers

Register via YAML (Section 4.3) or API (Section 7). Any string keyword can be a trigger. Multiple custom Amplifiers can coexist — but only one may be active at a time (stack depth: 1).

12.4 Webhook Support (Planned v1.1)

Tag switch events, Amplifier activations, and session expiry will emit webhooks to a registered endpoint. Subscribe via `POST /webhooks` (not yet available in v1.0). Design your consumer service to accept these event types early if you plan to integrate deeply.

13. Troubleshooting

Symptom	Likely Cause	Fix
401 on all requests	JWT expired or wrong secret	Re-generate token; verify <code>JWT_SECRET</code> matches
Router always returns default tag	Keywords not in <code>tags.yaml</code>	Check <code>trigger_keywords</code> ; run <code>seed_tags.py</code>
<code>AMPLIFIER_STACK_OVERFLOW</code>	Two amplifiers triggered simultaneously	Deactivate current amplifier before triggering another
<code>SESSION_EXPIRED</code> immediately	<code>ttl_seconds</code> too low	Increase <code>ttl_seconds</code> in <code>POST /sessions</code>
<code>LLM_BACKEND_TIMEOUT</code>	API key invalid or LLM quota exhausted	Check API key; verify quota in provider dashboard
<code>TAG_ISOLATION_BREACH</code>	Custom Tag prompt references another Tag	Remove cross-tag references from <code>system_prompt</code>
<code>PROMPT_INJECTION_DETECTED</code>	Input contains injection patterns	Review <code>user_input</code> ; do not include prompt override instructions
Snapshot restore fails	Snapshot was auto-generated (immutable)	Only manually created snapshots are restorable via API
High latency (> 5s)	Live LLM + no streaming	Enable <code>stream: true</code> ; switch LLM region closer to deployment
Context not restored after switch	Persistence Layer unreachable	Check Redis connection; verify <code>REDIS_URL</code>

Debugging Router Decisions

Always use the dry-run endpoint first before involving the LLM:

```
curl -X POST $BASE/diagnostics/route \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"user_input": "your input here", "current_tag": "auron"}' # Inspect:
trigger_type, routing_confidence, conflict, matched_keywords
```

✓ TIP:

Set

```
LOG_LEVEL=DEBUG
```

in your

```
.env
```

and watch

```
docker-compose logs -f api
```

to trace every routing decision, amplifier event, and persistence operation in real time. This resolves 90% of integration issues within minutes.

14. Contributing

14.1 Development Setup

```
# Install with dev dependencies pip install -e ".[dev]" # Install pre-commit
hooks pre-commit install # Run linting ruff check src/ tests/ mypy src/ #
Run formatting black src/ tests/
```

14.2 Contribution Workflow

```
1. Fork the repository 2. Create a feature branch: git checkout -b
feature/your-feature-name 3. Write code + tests (P0 and P1 coverage required
for new features) 4. Run the full test suite: pytest tests/ -v 5. Verify
coverage: pytest --cov=src (must be >= 85%) 6. Open a Pull Request
with: - Description of change - Test cases added (TC-[LAYER]-[NNN]
format) - Any new configuration options documented
```

14.3 Contribution Rules

- **New Tags:** Must include `system_prompt`, `trigger_keywords` (≥ 3), `isolation_level`, and at least 3 unit test cases
- **New API endpoints:** Must include schema, error codes, and integration tests
- **Security changes:** Require security test coverage (TC-SEC) and a security review note in the PR

- **No direct commits to main:** All changes via PRs; minimum 1 reviewer approval required

14.4 Code Style

Concern	Tooling / Convention
Python style	PEP 8 via <code>ruff</code> + <code>black</code>
Type hints	Required on all public functions (<code>mypy</code> strict mode)
Docstrings	NumPy style on all public classes and methods
Test naming	<code>test_{what}_{condition}</code> e.g. <code>test_router_detects_explicit_trigger</code>

15. Document Suite

The full companion document suite is located in `docs/`:

Document	File	Audience	Key Content
Architecture Specification v1.0	<code>architecture-spec-v1.0.docx</code>	Architects, Senior Engineers	System design, prompt stacks, rules, IAL / AMS / CAG detail
API Blueprint v1.0	<code>api-blueprint-v1.0.docx</code>	Backend Engineers, Integrators	30+ REST endpoints, schemas, auth, error codes
Test Suite Specification v1.0	<code>test-suite-spec-v1.0.docx</code>	QA, DevOps, Security	100+ test cases: unit → E2E → security → performance
Whitepaper v1.0	<code>whitepaper-v1.0.docx</code>	CTOs, Product Leaders	Strategic overview, use cases, competitive positioning
Developer README v1.0	<code>README.md</code>	Developers (all levels)	This document

16. License

MIT License Copyright (c) 2026 Emanuel (Auron) – Multi-Tag Agent Project
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Multi-Tag Agent System

.

Developer README v1.0

June 17, 2026

.

Emanuel (Auron)

.

MIT License

github.com/auron/multi-tag-agent (internal)

